

INF1103: Lab 2

Flow Control

Introduction

Welcome to Week 2! In this lab, we shift from sequential code to **Flow Control**. Programs rarely run in a straight line; they must make decisions (Conditionals) and repeat processes (Loops) based on user input. In this lab, you will shatter the "works on my machine" barrier.

Remember, the focus of this module is all about bridging logical programming constructs with modern, automated operational practices. We will systematically evolve our applications week by week, refactoring and packaging our code to fit enterprise-grade deployment standards.

The expectation is that you complete and verify your lab by the end of your scheduled lab session. True engineering mastery comes from hands-on problem-solving. Ensure you work through and complete each step of this lab by yourself. While your peers are excellent resources for discussing conceptual architectures, they must not write code for you or share their files with you!

It is highly critical that you have watched and engaged with the Week 2 lecture lessons before attempting this lab. Do not hesitate to ask plenty of questions to your professor and teaching assistants when you hit an infrastructure error.

Objectives

In this lab, you will:

1. **Understand and apply flow control concepts** by using conditional statements (if, elif, else) and loops (while) to control program execution.
2. **Implement input validation and error handling** to ensure programs can safely process user input and respond appropriately to invalid data.
3. **Gain hands-on experience with Docker** by setting up Docker Desktop, running Python applications in containers, and using volume mounts to execute local scripts within a containerized environment.

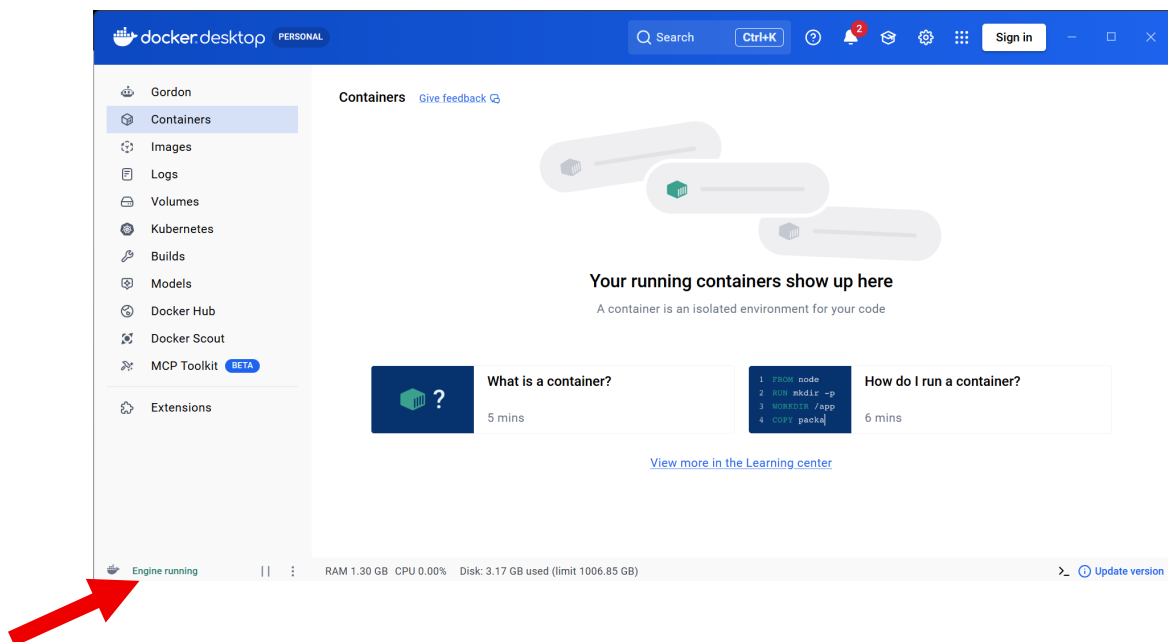
Remember: - Keep your code clean and your logic modular.

- Test your code against "invalid" inputs to ensure it remains stable.
- Work through the logic yourself. Your peers can explain concepts, but the keyboard time is yours.

Setting Up a Docker Environment

Docker is the containerization engine that solves the *"works on my machine"* problem by creating isolated sandboxes for your code.

1. Go to the official download page: docker.com/products/docker-desktop/.
2. Download and run the installer for your system.
3. **During installation:**
 - **Windows Users:** You will see a prompt asking to enable **WSL 2 (Windows Subsystem for Linux)**. Keep this option **checked**. It is critical for Docker's performance on Windows.
 - If prompted, restart your computer to finalize installation.
4. Launch the **Docker Desktop** application. Keep it running in the background. You will know it is ready when the status icon in the bottom-left corner turns **green (Running)**.



Let's confirm that your computer is communicating with the Docker engine successfully

1. In VS Code, open a new terminal window by clicking **Terminal -> New Terminal** from the top menu.
 - Type the following command and press Enter:

```
docker --version
```

- *Expected Output:* You should see version details like Docker version 24.x.x, build ...
2. To check if Docker is running, in the Terminal type:

```
docker ps
```

Assignment

This week, you will build the "**Smart Inventory Auditor**". This application mimics a real-world system where an operator processes daily stock deliveries. Your task is to ensure the system is robust, handles "dirty" user data without crashing, and respects business rules.

1. The Scenario

A store manager needs to process a stream of daily deliveries. Staff often make errors (entering text instead of numbers, negative values, or exceeding storage capacity).

Your job is to build the software that audits these entries in real-time.

Requirements

In your IDE, create a new file named `auditor.py`. Your program must:

1. Initialize the inventory to zero in the start
2. Run in a continuous loop asking user to enter a stock quantity, until the user types quit. (Think of which loop might be helpful here: for or while)
3. Accept stock values as integers.
4. **Handle invalid input:** If the user enters a string (e.g., "ten"), reject it, print an error, and move to the next iteration. (Hint: use `isdigit()`).
5. **Enforce business rules:** Reject negative numbers.
6. **Manage State:** Keep a running total of the inventory.
7. **Trigger Overstock Alert:** If the total inventory exceeds **500 units**, print an alert and break the loop immediately. (keep in mind of the conditional flow we discussed this week: if, elif and else)
8. **Reporting:** When the user types quit, print the *Total Units Processed* and the *Number of Failed/Rejected Entries*.

2. Version Control

- a. Before you begin writing logic, remember to keep your work backed up.
- b. Use git commands discussed last week to save your starting point.
- c. Create a new empty repository on GitHub e.g., inf1103-labs
- d. In the VS Code terminal, link the local repository to the remote URL:

```
git remote add origin https://github.com/<username>/<repo>.git
```

To generate a token in github:

- i) Sign in to GitHub.
 - ii) Click your profile picture and select Settings.
 - iii) In the left menu, click Developer settings.
 - iv) Open Personal access tokens > Fine-grained tokens.
 - v) Click Generate new token.
 - vi) Enter a token name and expiration date.
 - vii) Select repository access and permissions (Contents: Read and Write, Metadata: Read for typical Git usage).
 - viii) Click Generate token.
 - ix) Copy and securely store the token. You may not be able to view it again.
 - x) When prompted for a GitHub password in Git or VS Code, paste the token instead.
- e. You can Verify the remote connection:

```
git remote -v
```

- f. Push the local master branch commits to the remote repository for the first time:

```
git push -u origin master
```

- g. For subsequent commits made throughout the lab tasks, push updates using:

```
git push
```

- h. Instead of creating feature branches, you should commit your code changes locally as you complete each requirement of the lab sheet.
- i. Commit regularly as you implement each requirement of the lab.
- j. Once tested, then push those updates to GitHub repository

3. Mount and Run

We need to tell Docker to temporarily "link" your local folder inside the running container so the container can read and run your script.

Run the appropriate command below, navigate to the folder where you stored auditor.py in VS Code terminal:

```
docker run --rm -it -v "${PWD}:/usr/src/app" -w /usr/src/app python:3.11-slim python auditor.py
```

Understanding the command:

- **--rm**: Automatically destroys and cleans up the container once it finishes running your script.
- **-it**: This starts a Bash shell inside the container and gives you an interactive terminal where you can type commands.
- **-v \${PWD}:/usr/src/app** or **-v "\$(pwd)"/usr/src/app**: **Volume Mount**. Maps your current local directory (pwd) directly inside the container at the /usr/src/app directory path.
- **-w /usr/src/app**: **Working Directory**. Forces the container's shell to start execution directly inside the mapped folder.
- **python auditor.py**: Instructs the container to run our local Python script.

4. Create your first Custom Docker Blueprint

Instead of mounting a local directory every time, let's learn how to package your auditor.py script *directly inside* a custom Docker Image. This is the foundation of DevOps deployment. Let us create a Dockerfile:

- a. In your VS Code workspace, create a new file named Dockerfile.
 - **Note**: The file name must have no file extension (Do not name it Dockerfile.txt).
- b. In the Dockerfile add the following commands:

```
FROM python:3.11-slim
```

Use the official Python runtime image as your baseline kitchen

```
WORKDIR /app
```

Establish an active working directory inside the container

```
COPY auditor.py .
```

Copy our local auditor.py script into the container workspace

```
CMD ["python", "auditor.py"]
```

Define the execution command to run when the container

- c. Save the file

5. Build Your Custom Image

- Compile the Dockerfile into a reusable, unchangeable image blueprint.
- Run this command in your terminal:

```
docker build -t inf1003-labs-smart-auditor .
```

- `docker build`: Read the local Dockerfile and compile a new image.
- `-t inf1003-labs-smart-auditor`: Tags (names) your custom image so you can easily reference it.
- `.`: The dot is crucial! It tells Docker to look for the Dockerfile inside your current directory.

- Expected output:

```
[+] Building 1.2s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 404B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 32B
=> [1/3] FROM docker.io/library/python:3.11-slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca
=> => resolve docker.io/library/python:3.11-slim@sha256:ae52c5bef62a6bdd42cd1e8dffef86b9cd284bde9427da79839de7a4b983e7ca
=> CACHED [2/3] WORKDIR /usr/src/app
=> CACHED [3/3] COPY auditor.py .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:ace51ad7afafda2c192bff7ec1062250ecf2a8eef26efecdd232e40e596a2a7
=> => exporting config sha256:f00bad0db1db441d2a6a3916b156fe3fd966cdb29825e45e3825f6ee09e403b7
=> => exporting attestation manifest sha256:d14c9a0fc7be82bf5e29d3be892c17a01ec961106c5c5199e29d6c09dabaa46f
=> => exporting manifest list sha256:3dea23a67c8819cd3675d6298113b241566631c3a55f00e747941a1736bf7b7c
=> => naming to docker.io/library/inf1003-labs-smart-auditor:latest
=> => unpacking to docker.io/library/inf1003-labs-smart-auditor:latest
```

6. Run Your Custom Container

- Now that your custom image is built, anyone in the world can run your application identically with a single command. Run your compiled image:

```
docker run --rm -it inf1003-labs-smart-auditor
```

- `-i` = keep standard input open.
 - `-t` = allocate a terminal.
 - `(-it)` allow you to type values into input()
- Your application is officially containerized! You can run the code and see the output

7. Self-Reflection Task

Once your auditor is working, consider this:

"If I want to save the final inventory count to a file so it doesn't vanish when the container stops, what fundamental limitation of Docker are we hitting?"

What you need to submit?

1. Your python file (auditor.py)
2. Your git log. Store it in a text file.
`git log >log.txt`
3. A link to their remote GitHub repository showing the pushed commit history.
4. Docker Mount and run. A screenshot of the successful execution is usually sufficient.
5. Run the following command for evidence that the image was successfully built. Screenshot of the output.

`docker images`

6. Proof that the image runs successfully with a screenshot showing the program accepting input and producing results.
7. Put everything in a folder. Zip it and upload to xSite.